ELM327 / CAN Log Insights
How the 'working' app sets IDs and gets data (and what that implies for ObdInsight)

Generated 2026-01-21

This summary distills the key behavioral patterns observed in the provided ELM327 communication log from a functioning app, emphasizing CAN ID configuration, receive filtering, and multi-frame diagnostic exchanges. It is intended to guide ObdInsight's LeafAze0Contexts + ElmSession setup and to reframe expectations about when "monitoring" (sniffing) is actually necessary.

Key observation: the app is polling ECUs, not passively sniffing the bus

Across the log, the app repeatedly sets a transmit CAN ID, applies a receive filter for the matching response ID, and sends diagnostic requests (UDS/ISO-TP style). It obtains data by targeted request/response conversations with specific modules.

How IDs are used: request/response pairs

The dominant configuration pattern is:

Command	Meaning (practical)	Typical value
ATSH hhh	Set transmit CAN ID for subsequent requests (11-bit ID).	ATSH743
ATCRA hhh	Set receive filter: only forward frames whose CAN ID equals hhh.	ATCRA763
ATFCSH hhh	Set CAN ID used for Flow Control frames (ISO-TP).	ATFCSH743
ATFCSM 1	Enable flow control mode to support multi-frame responses.	ATFCSM1
ATAR	Reset/clear address-related filtering state before reconfiguring.	ATAR
ATCEA	Disable extended addressing (baseline for most 11-bit ISO-TP).	ATCEA
ATST hh	Set timeout (hh * 4 ms). App uses aggressive values for probing.	ATST08

Example from the screenshot: ATSH743 (request ID) pairs with ATCRA763 (response ID) for the Meter (M&A;) module. This matches a common Nissan convention where ResponseID = RequestID + 0x20 for many modules.

What the app does for robustness

- Before switching ECUs, it resets state (ATAR, ATCEA) then applies fresh ATSH/ATCRA and flow-control settings.
- It proves ECU presence using safe probes that may yield negative responses (still proof-of-life).
- It uses aggressive timeouts and filtering to reduce bus noise and fail fast when an ECU does not respond.

Session control patterns: 0x10 C0 and 0x10 81

The log/screenshot include DiagnosticSessionControl (service 0x10) values that look non-standard. Two interpretations help avoid debugging the wrong layer:

1) 0x10 81 is often "default session" with "suppress positive response" set

In UDS, many services use a subfunction byte whose bit 7 indicates "suppress positive response". So 0x81 can be understood as 0x01 with bit7 set. The ECU may accept the request but intentionally omit a positive response.

2) 0x10 C0 appears OEM-specific (Nissan/Consult-style)

Some manufacturer services may require entering a manufacturer-specific session such as 0xC0. A trailing byte (e.g., "...01") can be an OEM parameter record; acceptance can vary by ECU.

Implications for ObdInsight contexts (LeafAze0Contexts + ElmSession)

When target values live behind a diagnostic conversation, a context should define: (a) TX/RX ID pair, (b) any session activation sequence, and (c) query set + response parsing. Monitoring can help with broadcast signals, but it is not the primary mechanism the working app uses to access hard-to-get module data.

Steering example: expecting frames 0x002 and 0x300

If steering data arrives as unsolicited broadcast frames, monitoring should work, but reliability depends on adapter filtering state. An active ATCRA filter can block unrelated frame IDs, and some adapters retain filter state across mode switches. A robust path is: clear filters, then apply narrow filtering for 0x002 and 0x300. If frames only appear when modules are awake, add minimal keep-alive/session activation independent of steering reads.

Prompt seed for an iterative 'Ralph loop'

```
Goal: Make ObdInsight reliably acquire Leaf ZE0 steering frames (0x002, 0x300) and other
hard-to-get module data.
Observed in working app: reset state (ATAR, ATCEA) -> set ATSH (TX) + ATCRA (RX filter)
-> enable ISO-TP flow control -> send UDS/ISO-TP requests.
Session nuance: 0x10 C0 may be OEM session; 0x10 81 is often default session with
suppress-positive-response bit.
Tasks: review LeafAze0Contexts + ElmSession mapping; fix filter reset/restore; add
keep-alive if modules must be awake; produce test scripts for both broadcast sniffing and
polled diagnostics.
```

Note: separate broadcast-frame acquisition from diagnostic polling in your design and tests.